Encrypted Network Traffic Classification for QUIC Protocol: A Transfer Learning Approach

Aman Ullah Bhuiyan[1], Ashikun Nabi[1], Raqeebir Rab[1](✉), Abderrahmane Leshob[2], Tasnim Mahmud[1], and Adil Ahmed Khan[1]

[1] Department of Computer Science and Engineering, Ahsanullah University of Science and Technology, Dhaka, Bangladesh
raqeebir.cse@aust.edu

[2] Department of Analytics, Operations, and IT, University of Quebec at Montreal, Montreal, Canada
leshob.abderrahmane@uqam.ca

Abstract. The proliferation of various encrypted protocols has resulted in a significant portion of network traffic being protected from conventional inspection methods, including deep packet inspection (DPI) and port-based inspection. Encryption improves security and privacy; however, it presents considerable challenges for network administrators and security professionals regarding the monitoring, management, and classification of network traffic. Various deep learning approaches have demonstrated high performance in the classification of encrypted network traffic, necessitating substantial amounts of labeled data. Labeled data of network traffic is rarely available or infrequently accessible. A previous research demonstrated the potential to eliminate the necessity for this extensive labeled data. However, they utilized a substantial number of packets, which may elevate the classification latency. This paper presents a transfer learning approach utilizing LSTM (Long Short-Term Memory) for classifying network traffic flows with minimal number of packets and also to eliminate the need of large labeled dataset. We conducted three distinct sub-flow extraction methods to perform encrypted network traffic classification in three different cases and also introduced unique packet selection strategy for the sub-flows to perform the classification with fewer packets. All three sub-flow extraction methods exhibit superior performance, even with a limited number of packets, which is crucial for minimizing classification latency.

Keywords: Transfer Learning · Sub-flow Extraction · Unique Packet Selection Strategy · Network Traffic Classification · LSTM · QUIC Protocol · Low Latency

1 Introduction

Network traffic classification refers to the systematic identification and categorization of network data packets or flows into specific classes or groups such as

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025
L. Barolli (Ed.): AINA 2025, LNDECT 249, pp. 26–38, 2025.
https://doi.org/10.1007/978-3-031-87775-9_3

application type–Google Drive or YouTube, traffic type–uploading or video, protocol type–TCP or FTP, UDP and human behavior, etc. Each classified traffic class can be treated differently to ensure efficient utilization and security, such as improving quality of service (QoS), detecting anomalies, user behavior profiling, and performance optimization etc. Because of these different applications, protocols, and traffic types, network traffic classification becomes challenging. Moreover, the advent of different encrypted protocols such as HTTPS [5], HTTP/2 [6], QUIC [7] and broad use of these encrypted protocols in network communication increases the complexity of network traffic classification. These encrypted protocols, such as QUIC, are widely adopted for secure communication. According to [9], around 95% of websites run the HTTPS protocol, with more than half running the advanced HTTPS/2 version. According to [10], in the year 2021, nearly 99% of browsing time on Google Chrome involves HTTPS, with encryption rates of 95% on Android and 97% on Windows. According to [16], 8.4% of all the websites used QUIC, a modern transport layer protocol designed by Google for secure and efficient communication. Conventional machine learning (ML) techniques [8] depend on feature engineering, wherein important factors such as packet size, timing information, protocol type, and payload attributes etc. are manually extracted from raw data. These characteristics can be readily derived from unencrypted traffic. Nonetheless, encrypted protocols conceal the payload and occasionally the headers, resulting the critical informations such as URLs, hostnames and other application identifiers indecipherable. Furthermore, encrypted traffic may utilize non-standard ports, rendering port-based classification ineffective. Consequently, conventional machine learning methods are inadequate for classifying encrypted network traffic.

Deep learning methodologies have effectively applied to classify encrypted traffic into categories including web browsing, video streaming, and VoIP etc. by examining packet-level characteristics such as flow size, inter-arrival times, and direction, leveraging the automatic feature selection and intricate pattern recognition capabilities of these techniques. These approaches do not necessitate the decryption of the payload, making them appropriate for situations involving encrypted data. However, these deep learning algorithms necessitate a substantial quantity of labeled data. Labeling a network traffic dataset is a more expensive and arduous procedure in the presence of encrypted traffic, as encryption renders Deep Packet Inspection (DPI) based labeling unfeasible [8]. Conversely, a substantial amount of unlabeled data is accessible on the internet. In this paper, we have implemented a transfer learning strategy, wherein a deep learning model was used both in the base network and the target network. We employed the fundamental concept of transfer learning in our study, where a large unlabeled dataset was used to train a LSTM based base network and a small labeled dataset was used to train a LSTM based target network model. Thus we utilized the substantially available unlabeled data and rarely available labeled data of encrypted network traffic. Another purpose of this paper is to classify the network traffic with as less packets as possible for ensuring the classification in real time and minimum latency. The classification of encrypted traffic is executed by extract-

ing sub-flows containing varying packet quantities from diverse segments of the overall traffic. In this study, we implemented three distinct sub-flow extraction techniques to achieve three specific objectives in network traffic classification or to do the classification in three different cases. We have also implemented a unique packet selection approach to extract sub-flows from traffic flows. The primary objective of this study is to utilize unlabeled data and to classify network flow using sub-flow of initially selected n number of packets and further selected m number of unique packets from those initially selected n number of packets. The suggested three sub-flow extraction methods are as follows:

1. Sub-flow of First n Packets
2. Sub-flow of m Consecutive Packets
3. Sub-flow of m Sampled Packets

Upon thorough analysis, we found that all three techniques of our sub-flow extraction outperformed or achieved nearly identical accuracy to prior research, even with sub-flow of less number of packets. Among the three proposed sub-flow extraction method, the one with m sampled packets achieved the highest performance, attaining an accuracy of 98.77%.

2 Related Work

This section presents an analysis of studies pertaining to network traffic classification. In [2,3], the authors employed a sub-flow extraction technique to generate sub-flows consisting of the first n packets. In [2], the authors attained an accuracy of 90.67% by training a CNN model with the first 50 packets and in [3], 93.2% accuracy was achieved by training a KNN model with the first 5 packets for protocol level classification. Protocol classification is simpler than application classification because it typically involves identifying protocol specific headers where as application classification is more complex because it involves distinguishing between applications and operate under encryption. In [4], the author employed a sub-flow extraction method to generate sub-flows consisting of m consecutive packets. Their work entails classifying protocols, such as HTTP, SMTP, POP3, and IMAP, in unencrypted traffic, utilizing packet order, packet size, and packet direction. They assessed their model using 5 consecutive packets from 10 flows and achieved satisfactory accuracies(exactly not defined) for distinct protocol classes. The authors achieved 98% accuracy by training a CNN model using a sample of 45 packets in [1]. A network traffic classifier for IoT networks is presented in [13], integrating CNNs and RNNs to assess flow statistics while excluding payload or IP data, hence safeguarding privacy. Treating feature time series as pseudo-images improves detection accuracy, addresses imbalanced datasets, and necessitates little feature engineering, demonstrating efficacy in real-world IoT traffic management. In [14], the authors suggested a self-supervised learning approach for the classification of encrypted network traffic, necessitating minimum labeled data. The method attains elevated accuracy by utilizing extensive unlabeled datasets for pre-training and subsequently fine-tuning on a limited labeled dataset.

3 Dataset Processing

We utilized the QUIC dataset [12], developed by the researchers at UC Davis. They used scripts to obtain this dataset and additionally collected samples of human interactions. This dataset comprises five Google services: Google Drive, YouTube, Google Docs, Google Search, and Google Music. This dataset exclusively contains traffic of the QUIC protocol. The dataset contains a total of 6,589 traffic flows or text files. Each text file in the dataset contains all the packets of a single flow and each of these text files contains four columns: UNIX timestamp [15], relative time (from the initial packet in the flow), packet length (in bytes), and direction. Figure 1 illustrates the distribution of flows for each class.

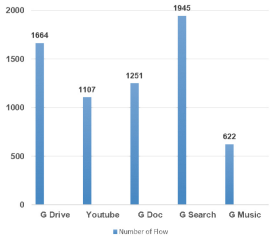

Fig. 1. Distribution of flows of QUIC dataset

3.1 Techniques for Subflow Extraction

We have employed a data augmentation strategy, wherein various quantities of sub-flows were extracted from the original flow. We have executed three distinct sub-flow extraction techniques to categorize encrypted network traffic in three different cases. This section provides a description of all three of these approaches. Let us define two variables, n and m, where n represents the quantity of initially picked packets for a sub-flow, and m denotes the count of extracted unique packets based on packet size from the initially selected n packets, with the condition that $m \leq n$.

1. **Sub-flow of First n Packets.** Real-time or online network traffic classification requires classifying the network flow with first few packets. Real-time or online refers to the classification of traffic as it traverses the network, rather than post-flow analysis. Prioritization, bandwidth allocation, application usage monitoring, network anomaly detection, and traffic-based resource allocation are possible with real-time classification. Therefore, classifying the flow from the start with minimal latency is necessary. So, we have selected the first n number of packets from the entire flow, starting from the first packet for classifying that flow.

2. **Sub-flow of m Consecutive Packets.** Consecutive packet selection is essential when the analyzer of the traffic flow may not be able to capture the entire flow but rather captures packets from the middle of the flow or captures a subset of the flow. Network congestion, limited recording capabilities, and particular monitoring requirements may cause this issue. To overcome this, we extract sub-flows by selecting consecutive packets from the beginning to the conclusion of the flow. For example, with a original flow of 1000 packets and a value of n = 25, the flow is partitioned into 40 sub-flows, each containing 25 packets. Additionally, these sub-flows are enhanced by choosing m unique packets according to packet size, where $m \leq n$. This enhances model efficacy. A threshold of 8 distinct packets is established, guaranteeing that a sub-flow is considered as valid and incorporated into the training set alone if it comprises a minimum of 8 unique packets.
3. **Sub-flow of m Sampled Packets.** Classifying a traffic flow by analyzing all packets is not efficient for the traffic analyzer. This may incur significant computational costs and delay the classification process. Rather than analyzing the complete flow, sub-flows are generated through packet sampling utilizing an incremental sampling technique, as outlined in [1]. This method includes three parameters: l, α, β. It selects n packets from a flow that are l packets apart from each other. After selecting β packets, l is multiplied by α. Reference [1] employed $l = 22$, $\alpha = 1.6$, and $\beta = 10$. In this study, packets were selected from a broader flow range with $l = 44$. We sampled as many as 100 sub-flows, determined by the overall flow length and originating from various locations. We selected m unique packets for a sub-flow from the initially selected n packets based on distinct packet size and also applied valid sub-flow criteria of minimum of 8 unique packets.

3.2 Data Preparation

This section describes the preparation of data for sub-flows in both the base and target networks, focusing on network traffic classification. Different types of input features can be utilized for network traffic classification, including header or payload-based features, statistical features, and time series features, among others [11]. Header or payload-based features rely on unencrypted data fields or messages. However, since our dataset pertains to the QUIC encrypted protocol, there are fewer unencrypted data fields and packets exchanged, complicating the classification process using header or payload-based features. In the first and second sub-flow extraction methods, it is assumed that the complete traffic flow is not available during classification. Consequently, the statistical features are not applicable. Therefore, three-time series features-inter-arrival time, packet size, and direction-were utilized from our dataset to construct these sub-flows.

The inter-arrival time and packet size were considered as features in these sub-flows. The packet length was multiplied by either $+1$ or -1, depending on the direction indicated by 0 or 1. A positive packet length indicates that the packet is traveling in the forward direction (from source to destination), while a negative packet length signifies that the packet is moving in the backward direction (from

destination to source). Upon integrating the two features, namely packet size and direction, from the dataset, we derived two sub-flow features: inter-arrival time and the product of packet size and direction. The features of these sub-flows were normalized by setting 1434 as the maximum packet size and 1 s as the maximum inter-arrival time. Zero padding was applied at the beginning of a sub-flow if it did not contain unique m number of packets. Let us consider a sub-flow in which x unique packets are obtained from n initially selected packets. If $x < m$, then zero padding was applied to first $m - x$ packets.

4 Methodology

Our approach utilizes transfer learning to classify network traffic, comprising two networks: a base network trained on a large unlabeled dataset and a target network trained on a small labeled dataset. The base dataset *Dbase* serves as the foundational dataset for training the base network, while *Dtarget* functions as the target dataset for training the target network. The target network generally shares the initial L_i layers with the base network, where L_i is less than L_l. The learnable parameters of the initial layers (specifically from 1 to L_i, where L_i is less than L_l) are derived from the base network and remain unchanged throughout the training of the target network. Training is conducted exclusively on the final and newly added layers, specifically from layer L_i to layer L_l. To ensure a fair comparison with [1], we constructed our *Dbase* by including a total of 6439 flows, with each class having the number of flows reduced by 30, as illustrated in Fig. 2. We constructed *Dtarget* by including 150 flows, with each class comprising 30 flows. In this study, we trained the target network using 20 labeled flows and validated it with the remaining 10 flows. Five-fold cross-validation was conducted on *Dtarget* to reduce the influence of data distribution on model performance.

4.1 Base Network

The base network is designed to extract significant feature representations from the unlabeled dataset, which can subsequently be utilized by the target network during the evaluation of the small labeled dataset. The network was trained using a Long Short-Term Memory (LSTM) layer in conjunction with a fully connected architecture. A bidirectional LSTM layer was employed to capture both forward and backward dependencies in the input sequence, while two linear layers were utilized to generate output predictions. The hidden state (h_0) and cell state (c_0) of the LSTM layer were initialized to zero at the start of each forward pass to maintain uniform starting conditions across all training samples. The base network model architecture is illustrated in Fig. 2.

The Base Network model receives extracted sub-flows from Dbase as input. We cannot utilize the actual labels to train our model though the labels are available as we want to train the model utilizing unlabel data. The target of the regression function for the Base Network requires adjustment. Statistical features

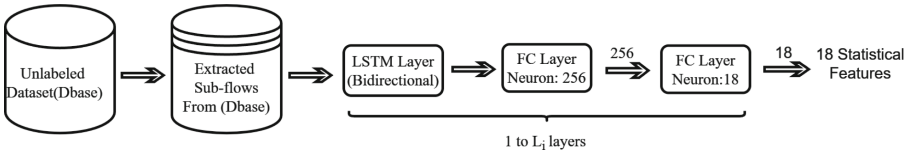

Fig. 2. Base network model architecture

obtained from entire network traffic flow can reveal significant aspects regarding the overall behavior and structure of the traffic. These features represent both temporal and spatial aspects of flows, including the frequency of packet transmission and the size of payloads. These characteristics are sufficiently distinct to distinguish between applications or classes. Additionally, statistical features are independent of specific packet content, rendering them useful for classifying encrypted traffic where packet payloads are not accessible. Eighteen statistical features were utilized as the regression target in the training of the base network. Eighteen statistical features were derived from the original flow characteristics, specifically inter-arrival time, packet size, and direction. The 18 statistical features are presented in Table 1. At this stage, the model will be trained to approximate the 18 statistical features using the extracted sub-flows from that flow as input. The base network was trained using NVDIA GeForce RTX 3090 in conjunction with the SmoothL1 loss function, which exhibits reduced sensitivity to outliers compared to mean squared error (MSE), the Adam optimizer (Adaptive Moment Estimation), learning rate 0.0005 and batch size 64 for 300 epoch.

Table 1. List of 18 Statistical Features

1	Difference of maximum and minimum packet size in forward direction
2	Median of packet size in forward direction
3	Coefficient of variation of packet size in forward direction
4	Difference of maximum and minimum packet size in backward direction
5	Median of packet size in backward direction
6	Coefficient of variation of packet size in backward direction
7	Difference of maximum and minimum packet size in both direction
8	Median of packet size in both direction
9	Coefficient of variation of packet size in both direction
10	Difference of maximum and minimum inter arrival time in forward direction
11	Median of inter arrival time in forward direction

continued

Table 1. continued

12	Coefficient of variation of inter arrival time in forward direction
13	Difference of maximum and minimum inter arrival time in backward direction
14	Median of inter arrival time in backward direction
15	Coefficient of variation of inter arrival time in backward direction
16	Difference of maximum and minimum inter arrival time in both direction
17	Median of inter arrival time in both direction
18	Coefficient of variation of inter arrival time in both direction

4.2 Target Network

Following the training of the model using the *Dbase* dataset, we transferred the learned weights of the LSTM layers to a new model and modified the architecture of the fully connected layers by incorporating three additional linear layers alongside the existing ones as illustrated in Fig. 3. At this stage, the LSTM layer is frozen and only the linear layers are trained. During this phase, we conducted supervised training using the small labeled dataset *Dtarget*. The model receives extracted sub-flows from the labeled dataset *Dtarget* as input and predicts the actual class of the traffic flows. We have employed Cross Entropy Loss as the loss function and the Adam Optimizer in target network. The learning rate, batch size and GPU is same as the base network.

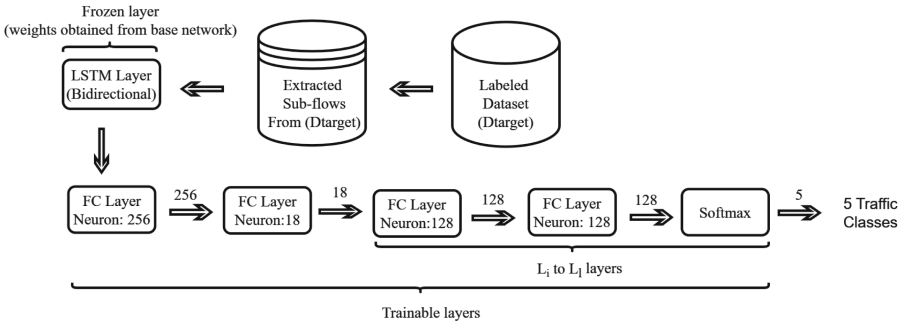

Fig. 3. Target network model architecture

5 Performance Analysis

The network traffic classification framework has been assessed for the three proposed sub-flow selection methods: Sub-flow with First n Packets, Sub-flow with m Consecutive Packets, and Sub-flow with m Sampled Packets. We evaluated the model for each sub-flow extraction technique using different combinations

of the values of n and m to determine the optimal combination. The model's accuracy has been assessed through macro averaging for each combination. Four performance metrics were calculated for each class: accuracy, precision, recall, and F1 score.

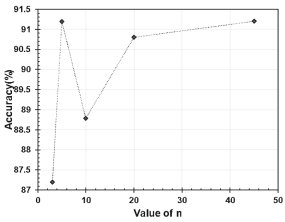

Fig. 4. Accuracy of the model for different values of n

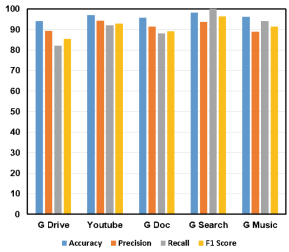

Fig. 5. Individual class performance of the model for n = 5

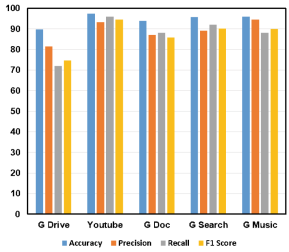

Fig. 6. Individual class performance of the model for n = 3

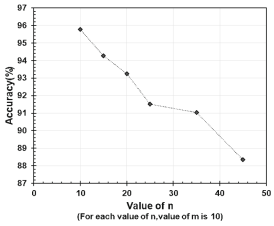

Fig. 7. Accuracy of the model for different values of n and m = 10

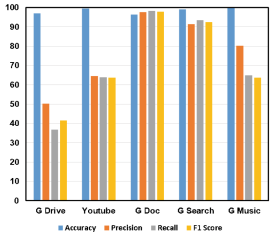

Fig. 8. Individual class performance of the model for n = 10, m = 10

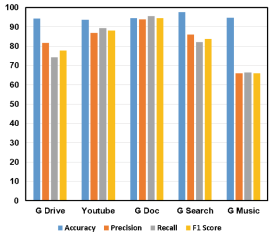

Fig. 9. Individual class performance of the model for n = 45, m = 10

5.1 Performance Analysis of Sub-flow with First n Packets

Sub-flow with First n Packets involves selecting only the first n packets to form a sub-flow from a entire flow. From the base dataset *Dbase*, we obtained 6439 sub-flows corresponding to 6439 flows, and from the target dataset *Dtarget*, we derived 150 sub-flows from 150 flows. The model was evaluated 45, 20, 10, 5, and 3 packets as value of n, where n represents the number of initially selected packets for a sub-flow. Figure 4 illustrates the accuracy of the model across various values

of n. The highest accuracy achieved was 91.19% for the first five packets, while the lowest accuracy recorded was 87.2% for the first three packets. Figures 5 and 6 illustrate the performance of individual classes in our model evaluation using 5 packets and 3 packets, respectively.

5.2 Performance Analysis of Sub-flow with m Consecutive Packets

The model was evaluated with sub-flows of consecutive packets using various values of n: 45, 35, 25, 20, 15, and 10, while maintaining a fixed value of m at 10, and threshold of minimum unique packets in a subflow at 8. The procedure entailed the selection of m unique packets from a total of initially selected n consecutive packets, incorporating zero padding when the number of unique packets was fewer than 10. For n = 25, if only 8 unique packets exist, the initial 2 packets will be padded with zeros, while the features of the remaining 8 packets will be utilized. Figure 7 illustrates the accuracy of our model for various values of n, with m set to 10. Fixing m at 10, a decrease in n correlated with an increase in accuracy, while an increase in n was associated with a decrease in accuracy. A highest accuracy of 95.78% was attained for n = 10 and m = 10. The minimum accuracy recorded was 88.34% for n = 45 and m = 10. To achieve maximum accuracy (n = 10), a total of 311,732 sub-flows were extracted from the base dataset *Dbase*, while 7,316 sub-flows were derived from the target dataset *Dtarget*, averaged across five-fold cross-validation. At the lowest accuracy (n = 45), a total of 146,783 sub-flows were derived from *Dbase*, while 3,496 sub-flows were obtained from *Dtarget*. Figures 8 and 9 illustrate the individual class performance of the two combinations of the model, respectively.

5.3 Performance Analysis of Sub-flow with m Sampled Packets

Here, n initial packets for a sub-flow were selected using incremental sampling ($l = 44$, $\alpha = 1.6$, and $\beta = 10$), followed by the selection of m unique packets. Zero padding is also implemented at the beginning. The model was evaluated by integrating various values of n and m, as presented in Fig. 12, along with their associated accuracy metrics. The highest accuracy achieved was 98.77% for n = 30 and m = 15. In this combination, we obtained 146,857 sub-flows from the base dataset *Dbase* and 3,304 sub-flows from the target dataset *Dtarget* (average of five-fold cross-validation). The lowest accuracy achieved was 96.07% for n = 80 and m = 25. In this combination, we obtained 203,981 sub-flows from the base dataset *Dbase* and 4,705 sub-flows from the target dataset *Dtarget*, based on an average of five-fold cross-validation. The individual class performance of our model for these two combinations is presented in Figs. 10 and 11 respectively.

The comparison summary of three distinct Sub-flow extraction techniques is presented in Table 2.

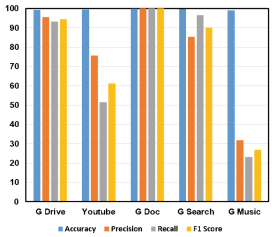

Fig. 10. Individual class performance of the model for n = 30, m = 15

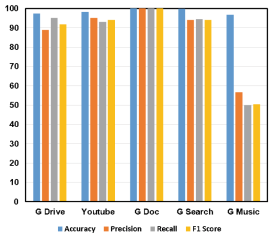

Fig. 11. Individual class performance of the model for n = 80, m = 25

Value of n	Value of m	Accuracy (%)
80	25	96.07
75	25	96.67
35	25	97.43
30	25	98.63
30	15	98.77
30	10	98.09

Fig. 12. Accuracy of the model for different values of n and m

5.4 Comparison with Other Network Traffic Classification Models

A CNN model in [2] obtained 90.67% accuracy with first 50 packets. Our LSTM model trained on a sub-flow of first 5 packets achieved 91.19%, showing that fewer packets can still perform well. Our LSTM model achieved 98.77% accuracy with 30 initially selected packets through incremental sampling and further selected 15 unique packets based on distinct packet size from initially selected 30 packets, compared to [1] which reported 98% accuracy with 45 packets. In [4], the author employed a sub-flow extraction method to generate sub-flows consisting of 5 consecutive packets from 10 flows and achieved satisfactory accuracy (not exactly defined). Our work achieved a maximum accuracy of 95.78% by training the model with a sub-flow of an originally selected set of 10 consecutive packets and further selecting 10 unique packets from this initial set.

Table 2. Summary of performance analysis and required time for model evaluation

Sub-flow extraction methods	Sub-flow with first few packets		Sub-flow with consecutive packets		Sub-flow with sampled packets	
Accuracy	Highest Accuracy (%)	Lowest Accuracy (%)	Highest Accuracy (%)	Lowest Accuracy (%)	Highest Accuracy (%)	Lowest Accuracy (%)
Value of Accuracy	91.19	87.2	95.78	88.34	98.77	96.07
Value of n	5	3	10	45	30	80
Value of m	-	-	10	10	15	25
Number of sub-flows for base network	6439	6439	311732	146783	146857	203981
Number of sub-flows for target network	150	150	7316	3496	3304	4705
Required time for base network (seconds)	257.723	259.165	3375.467	1683.938	1596.137	2200.998
Required time for target network (seconds)	64.091	63.456	628.852	294.661	300.741	416.973

6 Conclusion and Future Work

This paper presents a transfer learning approach utilizing LSTM for the classification of encrypted network traffic associated with the QUIC protocol, demonstrating high accuracy with a reduced number of packets across three distinct sub-flow selection scenarios. Our approach shows that employing a single LSTM layer can yield high accuracy, even when working with fewer packets and a limited amount of labeled data. Our objective is to illustrate the potential performance improvements that can be achieved through the integration of our distinctive packet selection strategy, even when utilizing a minimal quantity of packets. Figure 1 illustrates that certain classes exhibit a lower number of flows, which suggests an imbalance within the dataset *Dbase*. We refrained from implementing any balancing techniques on the dataset to evaluate the performance of our approach using the original, unaltered dataset. The precision of the 2nd and 3rd sub-flow extraction methods reduces as the value of n decreases. Achieving high precision involves a careful balance of the dataset and the extracted sub-flows. Studies [1,14] illustrate that transfer learning approaches yield effective results when a model is initially pre-trained on a distinct unlabeled dataset and subsequently re-trained on a different target dataset. The methodology we propose facilitates the development of extensive pre-trained models utilizing easily accessible unlabeled data pertaining to the QUIC protocol [17]. Subsequently, these models can be re-trained on smaller labeled datasets specific to individual organizations, enabling the classification of encrypted traffic flows with a reduced number of packets. This approach guarantees minimal classification latency across the three distinct scenarios detailed in the data preparation section. We intend to investigate these capabilities of our proposed method in subsequent research.

References

1. Rezaei, S., Liu, X.: How to achieve high classification accuracy with just a few labels: a semi-supervised approach using sampled packets. arXiv preprint arXiv:1812.09761 (2018)
2. Rezaei, S., Liu, X.: Multitask learning for network traffic classification. In: 2020 29th International Conference on Computer Communications and Networks (ICCCN), pp. 1–9. IEEE (2020)
3. Liu, Y., Chen, J., Chang, P., Yun, X.: A novel algorithm for encrypted traffic classification based on sliding window of flow's first N packets. In: 2017 2nd IEEE International Conference on Computational Intelligence and Applications (ICCIA), pp. 463-470. IEEE (2017)
4. Satoh, A., Osada, T., Abe, T., Kitagata, G., Shiratori, N., Kinoshita, T.: A classifiable sub-flow selection method for traffic classification in mobile IP networks. J. Inf. Process. Syst. **6**(3), 307–322 (2010)
5. HTTP Over TLS. https://datatracker.ietf.org/doc/html/rfc2818. Accessed 27 Nov 2024
6. Belshe, M., Peon, R., Thomson, M.: Hypertext transfer protocol version 2 (HTTP/2) (2015)

7. Iyengar, J., Swett, I., Hamilton, R., Wilk, A.: QUIC: a UDP-based secure and reliable transport for HTTP/2. In: Work in Progress, IETF (2015)
8. Velan, P., Čermák, M., Čeleda, P., Drašar, M.: A survey of methods for encrypted traffic classification and analysis. Int. J. Netw. Manag. **25**(5), 355–374 (2015)
9. SSL Stats for Secure Browsing in 2024. https://serpwatch.io/blog/ssl-stats/. Accessed 27 Nov 2024
10. New ThreatLabz Report Reveals Over 85% of Attacks Are Encrypted. https://www.zscaler.com/blogs/security-research/2022-encrypted-attacks-report/. Accessed 27 Nov 2024
11. Rezaei, S., Liu, X.: Deep learning for encrypted traffic classification: an overview. IEEE Commun. Mag. **57**(5), 76–81 (2019)
12. UCDavisQUIC: Netflow QUIC Dataset from encrypted traffic classification. https://www.kaggle.com/datasets/guillaumefraysse/ucdavisquic. Accessed 27 Nov 2024
13. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J.: Network traffic classifier with convolutional and recurrent neural networks for Internet of Things. IEEE Access **5**, 18042–18050 (2017)
14. Towhid, M.S., Shahriar, N.: Encrypted network traffic classification using self-supervised learning. In: 2022 IEEE 8th International Conference on Network Softwarization (NetSoft), pp. 366–374 (2022)
15. Unix Time Stamp. https://www.unixtimestamp.com/. Accessed 27 Nov 2024
16. Usage statistics of site elements for websites. https://w3techs.com/technologies/overview/site_element. Accessed 27 Nov 2024
17. Luxemburk, J., Hynek, K., Čejka, T., Lukačovič, A., Šiška, P.: CESNET-QUIC22: A large one-month QUIC network traffic dataset from backbone lines. Data Brief **46**, 108888 (2023)